Zelda API Data Pipeline

Requirements

- Python 3.13.12
- SQLite3
- Python Libraries: Pandas, SQLite3
- PowerBI Desktop
- VS Code with Jupyter Notebooks
- SQLite ODBC Driver (64bit)
- DB Browser for SQLite3 *(optional, for viewing data)*

Setup

1. Download and extract ZIP to any location
2. Execute Python notebooks in order:
    a. `1_extract_api_to_landing.ipynb`
    b. `2_landing_to_staging.ipynb`
    c. `3_staging_to_silver.ipynb`
    d. `4_staging_to_links.ipynb`
3. Open ODBC Data Source Administrator, go to **System DSN** and click **Add**
    a. Select SQLite3 ODBC Driver
    b. Point to `ZeldaDatabase.db`
4. Open `.pbix` file and connect via ODBC to `ZeldaDatabase`
    a. Windows authentication — no username/password required

Pipeline Architecture

Landing

- All API endpoints are extracted and saved as JSON files to the landing layer
- A metadata log is created per entity capturing:
    - Entity name
    - Start time, end time, duration
    - Status
    - Row count

    - Pages extracted
    - Filename
    - Source URL
    - Latest release date *(originally intended for watermark — see Future Considerations)*
- Request exceptions are handled for connectivity failures

1_extract_api_to_landing.ipynb ● | 2_landing_to_staging.ipynb ● | 4_staging_to_links.ipynb ● | 3_staging_to_silver.ipynb ●

Zelda ETL > Landing > 1_extract_api_to_landing.ipynb > import requests

Generate | + Code | + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

```python
    with open(output_file, "w", encoding="utf-8") as f:
        json.dump(all_data, f, indent=4)

    # Compute latest release date if 'released_date' exists
    latest_release_date_str = None
    if entity == "games" and all_data:
        df = pd.DataFrame(all_data)
        df["released_date"] = pd.to_datetime(df["released_date"], errors="coerce")
        latest_release_date = df["released_date"].max()
        latest_release_date_str = latest_release_date.strftime("%Y-%m-%d") if pd.notna(latest_release_date) else

    end_time = time.time()
    duration = round(end_time - start_time, 2)

    # Log metadata
    log = {
        "entity": entity,
        "start_time": start_timestamp,
        "end_time": datetime.now().isoformat(),
        "duration_seconds": duration,
        "status": "SUCCESS",
        "rows_fetched": len(all_data),
        "pages_fetched": page - 1,
        "filename": output_file.name,
        "source": f"{BASE_URL}/{entity}",
        "latest_release_date": latest_release_date_str
    }

    with open(LOG_FILE, "a", encoding="utf-8") as log_f:
        log_f.write(json.dumps(log) + "\n")

    print(f"Saved {len(all_data)} records to {output_file}")
    if latest_release_date_str:
        print(f"Latest release date: {latest_release_date_str}")
```

[1] ✓ 1m 45.0s

```
=== Fetching entity: games ===
Fetched page 1 (12 records)
Saved 12 records to landing\games_raw.json
Latest release date: 2020-11-20

=== Fetching entity: staff ===
Fetched page 1 (20 records)
Fetched page 2 (20 records)
Fetched page 3 (20 records)
Fetched page 4 (20 records)
Fetched page 5 (20 records)
Fetched page 6 (20 records)
Fetched page 7 (20 records)
Fetched page 8 (20 records)
Fetched page 9 (20 records)
Fetched page 10 (20 records)
Fetched page 11 (20 records)
Fetched page 12 (14 records)
Saved 234 records to landing\staff_raw.json
```

Staging

For each JSON file in landing:

- Table name is derived from the filename
- Data list is loaded from JSON
- Column list is extracted from the data
- Staging table is dropped if it already exists

- Table is recreated using the extracted column list
- Each record is inserted using parameterised column list

```python
        with open(file_path, "r", encoding="utf-8") as f:
            data = json.load(f)

            # Handle API wrapper
            if isinstance(data, dict) and "data" in data:
                data = data["data"]

            entries = data if isinstance(data, list) else [data]

        # Detect columns for THIS file only
        columns = set()
        for entry in entries:
            if isinstance(entry, dict):
                columns.update(entry.keys())

        columns = list(columns)

        # Drop table if exists (fresh staging load)
        cursor.execute(f'DROP TABLE IF EXISTS "{table_name}"')

        # Create table WITHOUT primary key
        columns_def = ", ".join([f'"{col}" TEXT' for col in columns])

        cursor.execute(f'''
        CREATE TABLE "{table_name}" (
            {columns_def}
        )
        ''')

        # Insert rows
        for entry in entries:
            if isinstance(entry, dict):
                values = [str(entry.get(col, None)) for col in columns]
                placeholders = ", ".join(["?"] * len(columns))

                cursor.execute(
                    f'INSERT INTO "{table_name}" ({", ".join(columns)}) VALUES ({placeholders})',
                    values
                )

        conn.commit()
        print(f" Loaded {len(entries)} rows into {table_name}")

    conn.close()
```

```
Loaded 255 rows into stg_bosses
Loaded 1657 rows into stg_characters
Loaded 335 rows into stg_dungeons
Loaded 12 rows into stg_games
Loaded 1849 rows into stg_items
Loaded 815 rows into stg_monsters
Loaded 1470 rows into stg_places
Loaded 234 rows into stg_staff
```

Silver (Staging to Silver)

- A SHA-256 HashDiff is computed on all non-key descriptive columns to detect record changes
- Only columns without foreign keys are loaded to silver to avoid duplication from one-to-many relationships
- SCD Type 2 logic is applied:
    - Existing active records where the HashDiff has changed are closed off by setting `ChangeEffectiveTo = DATETIME('now')`

- New or changed records are inserted with `ChangeEffectiveFrom = DATETIME('now')` and `ChangeEffectiveTo = '9999-12-31 23:59:59'`

```
1_extract_api_to_landing.ipynb   2_landing_to_staging.ipynb   4_staging_to_links.ipynb   3_staging_to_silver.ipynb
Zelda ETL > Landing > 3_staging_to_silver.ipynb > import sqlite3
Generate  + Code  + Markdown  | Run All  Restart  Clear All Outputs  | Jupyter Variables  Outline  ...  Python 3.13.

                sha256(COALESCE(stg.name,''))
            FROM "stg_staff" stg
            LEFT JOIN "silver_staff" ss ON stg.id = ss.Staff_ID AND ss.ChangeEffectiveTo = '9999-12-31 23:59:59'
            WHERE ss.Staff_ID IS NULL
              OR ss.HashDiff <> sha256(COALESCE(stg.name,''))"""
        ),
    ]

    total_updated = 0
    total_inserted = 0
    failed_tables = []

    for table_name, update_sql, insert_sql in tables:
        updated, inserted, success = execute_table_load(conn, update_sql, insert_sql, table_name)
        if not success:
            failed_tables.append(table_name)
        total_updated += updated
        total_inserted += inserted

    conn.close()

    summary = f"Pipeline complete | Total rows closed off: {total_updated} | Total rows inserted: {total_inserted}"
    print(summary)
    logger.info(summary)

    if failed_tables:
        warning = f"Tables that errored: {failed_tables}"
        print(warning)
        logger.warning(warning)
    else:
        success_msg = "All tables loaded successfully"
        print(success_msg)
        logger.info(success_msg)


run_silver_loads(r"C:\Users\liam_\Documents\Games DE Project\Zelda DB\ZeldaDatabase.db")
[4]  ✓ 0.0s  Python

2026-02-21 18:56:12,809 - INFO - Connected to: C:\Users\liam_\Documents\Games DE Project\Zelda DB\ZeldaDatabase.db
2026-02-21 18:56:12,815 - INFO - silver_games: 12 rows already in DB | no changes detected, rows already up to date
2026-02-21 18:56:12,825 - INFO - silver_character: 1657 rows already in DB | no changes detected, rows already up to date
2026-02-21 18:56:12,828 - INFO - silver_dungeons: 335 rows already in DB | no changes detected, rows already up to date
2026-02-21 18:56:12,831 - INFO - silver_Bosses: 255 rows already in DB | no changes detected, rows already up to date
2026-02-21 18:56:12,836 - INFO - silver_monsters: 815 rows already in DB | no changes detected, rows already up to date
2026-02-21 18:56:12,847 - INFO - silver_items: 1849 rows already in DB | no changes detected, rows already up to date
2026-02-21 18:56:12,859 - INFO - silver_places: 1470 rows already in DB | no changes detected, rows already up to date
2026-02-21 18:56:12,862 - INFO - silver_staff: 234 rows already in DB | no changes detected, rows already up to date
2026-02-21 18:56:12,863 - INFO - Pipeline complete | Total rows closed off: 0 | Total rows inserted: 0
2026-02-21 18:56:12,864 - INFO - All tables loaded successfully
Connected to: C:\Users\liam_\Documents\Games DE Project\Zelda DB\ZeldaDatabase.db
silver_games: 12 rows already in DB | no changes detected, rows already up to date
silver_character: 1657 rows already in DB | no changes detected, rows already up to date
silver_dungeons: 335 rows already in DB | no changes detected, rows already up to date
silver_Bosses: 255 rows already in DB | no changes detected, rows already up to date
silver_monsters: 815 rows already in DB | no changes detected, rows already up to date
silver_items: 1849 rows already in DB | no changes detected, rows already up to date
silver_places: 1470 rows already in DB | no changes detected, rows already up to date
silver_staff: 234 rows already in DB | no changes detected, rows already up to date
Pipeline complete | Total rows closed off: 0 | Total rows inserted: 0
```

Link Tables (Staging to Links)

- For each staging table, bridge/link tables are created to resolve many-to-many relationships
- Foreign key URL strings are parsed to extract GUIDs (the ID after the last `/` in the URL)
- Comma-separated values in a single column are split into individual rows
- Example: `stg_bosses` produces `link_boss_games` (Boss_ID, Game_ID) and `link_boss_dungeons` (Boss_ID, Dungeon_ID)
- `INSERT OR IGNORE` is used to prevent duplicates on re-runs

- These link tables are also used for data quality — where a Game_ID exists in a link table but has no matching record in `silver_games`, this surfaces as an orphaned record

```
1_extract_api_to_landing.ipynb   2_landing_to_staging.ipynb   4_staging_to_links.ipynb   3_staging_to_silver.ipynb
Zelda ETL > Landing > 4_staging_to_links.ipynb > import sqlite3
Generate  + Code  + Markdown  | Run All  Restart  Clear All Outputs  | Jupyter Variables  Outline  ...  Python 3.13.1

        cursor.execute('SELECT id, inhabitants FROM stg_places WHERE inhabitants IS NOT NULL')
        rows = [(place_id, char_id) for place_id, inhabitants in cursor.fetchall() for char_id in extract_guids(inhabitants)]
        inserted, success = execute_link_load(conn, cursor, "link_place_characters", rows, 'INSERT OR IGNORE INTO "link_place_ch
        total_inserted += inserted
        if not success: failed_tables.append("link_place_characters")

        cursor.execute('SELECT id, worked_on FROM stg_staff WHERE worked_on IS NOT NULL')
        rows = [(staff_id, game_id) for staff_id, worked_on in cursor.fetchall() for game_id in extract_guids(worked_on)]
        inserted, success = execute_link_load(conn, cursor, "link_staff_games", rows, 'INSERT OR IGNORE INTO "link_staff_games"
        total_inserted += inserted
        if not success: failed_tables.append("link_staff_games")

        conn.close()

        summary = f"Pipeline complete | Total link rows inserted: {total_inserted}"
        print(summary)
        logger.info(summary)

        if failed_tables:
            warning = f"Tables that errored: {failed_tables}"
            print(warning)
            logger.warning(warning)
        else:
            success_msg = "All link tables loaded successfully"
            print(success_msg)
            logger.info(success_msg)


    run_link_loads(r"C:\Users\liam_\Documents\Games DE Project\Zelda DB\ZeldaDatabase.db")

[1]  ✓ 0.1s                                                                                                  Python

...  2026-02-21 19:00:20,667 - INFO - Connected to: C:\Users\liam_\Documents\Games DE Project\Zelda DB\ZeldaDatabase.db
     2026-02-21 19:00:20,674 - INFO - link_boss_games: 419 rows already in DB | 0 rows inserted | 419 rows skipped (already existed)
     2026-02-21 19:00:20,677 - INFO - link_boss_dungeons: 212 rows already in DB | 0 rows inserted | 213 rows skipped (already exist
     2026-02-21 19:00:20,689 - INFO - link_character_games: 1974 rows already in DB | 0 rows inserted | 1974 rows skipped (already e
     2026-02-21 19:00:20,693 - INFO - link_dungeon_games: 441 rows already in DB | 0 rows inserted | 441 rows skipped (already exist
     2026-02-21 19:00:20,705 - INFO - link_monster_games: 1710 rows already in DB | 0 rows inserted | 1710 rows skipped (already exi
     2026-02-21 19:00:20,723 - INFO - link_item_games: 2503 rows already in DB | 0 rows inserted | 2503 rows skipped (already existe
     2026-02-21 19:00:20,737 - INFO - link_place_games: 1728 rows already in DB | 0 rows inserted | 1728 rows skipped (already exist
     2026-02-21 19:00:20,744 - INFO - link_place_characters: 765 rows already in DB | 0 rows inserted | 766 rows skipped (already ex
     2026-02-21 19:00:20,747 - INFO - link_staff_games: 314 rows already in DB | 0 rows inserted | 314 rows skipped (already existed
     2026-02-21 19:00:20,749 - INFO - Pipeline complete | Total link rows inserted: 0
     2026-02-21 19:00:20,749 - INFO - All link tables loaded successfully
     Connected to: C:\Users\liam_\Documents\Games DE Project\Zelda DB\ZeldaDatabase.db
```

Data Model

Dimension Views

Clean active record views (filtered to `ChangeEffectiveTo = '9999-12-31 23:59:59'`) for each silver table. Contain only descriptive columns, no foreign keys.

Note: 9999-12-31 filter could be removed should point in time analytice be required to show states of the data at different times when filtered/joined to other dimensions)

```sql

CREATE VIEW "vw_dim_monsters" AS
SELECT
    Monster_ID,
    Monster_Name,
    Monster_Description
FROM "silver_monsters"
WHERE ChangeEffectiveTo = '9999-12-31 23:59:59';


CREATE VIEW "vw_dim_items" AS
SELECT
    Item_ID,
    Item_Name,
    Item_description
FROM "silver_items"
WHERE ChangeEffectiveTo = '9999-12-31 23:59:59';


```

Find in editor Case Se

Analytical Views

Purpose-built views joining silver tables through link tables to answer the three task questions:

- `vw_analytical_staff_most_games` — Staff with a count of games worked on, grouped by Staff ID and Name
- `vw_analytical_monsters_min_3_games` — Monsters with an aggregated game count, filterable in Power BI to >= 3

- `vw_analytical_items_by_game_count` — Items with descriptions and game appearance counts

```sql
CREATE VIEW "vw_analytical_staff_most_games" AS
SELECT
    ss.Staff_ID,
    ss.Staff_Name,
    COUNT(lsg.Game_ID) AS Number_of_Games
FROM "silver_staff" ss
INNER JOIN "link_staff_games" lsg ON ss.Staff_ID = lsg.Staff_ID
WHERE ss.ChangeEffectiveTo = '9999-12-31 23:59:59'
GROUP BY ss.Staff_ID, ss.Staff_Name
ORDER BY Number_of_Games DESC;


-- 2. vw_analytical_monsters_games
CREATE VIEW "vw_analytical_monsters_games" AS
SELECT
    sm.Monster_ID,
    sm.Monster_Name,
    sm.Monster_Description,
    Number_of_Games.Number_of_Games,
    CASE when Number_of_Games >=3 then 1 else 0 end as MoreThan3GamesFlag
FROM "silver_monsters" sm
INNER JOIN (SELECT count(lmg.Game_ID) as Number_Of_Games, lmg.Monster_ID
            from link_monster_games lmg
            group by lmg.Monster_ID) as Number_of_Games on Number_of_Games.Monster_ID = sm.M
WHERE sm.ChangeEffectiveTo = '9999-12-31 23:59:59'
and Monster_Name = "Ache"



CREATE VIEW "vw_analytical_items_by_game_count" AS
SELECT
    i.Item_ID,
    i.Item_Name,
    i.Item_description,
    COUNT(lig.Game_ID) AS Number_of_Games
FROM "silver_items" i
INNER JOIN "link_item_games" lig ON i.Item_ID = lig.Item_ID
WHERE i.ChangeEffectiveTo = '9999-12-31 23:59:59'
GROUP BY i.Item_ID, i.Item_Name, i.Item_description
ORDER BY Number_of_Games DESC;
```

Data Quality Views

Views exposing orphaned link records — where a linked ID exists in a link table but has no matching record in the corresponding silver table:

- Example: `vw_dq_boss_game_orphans` — Bosses with a Game_ID in `link_boss_games` that does not exist in `silver_games`
- A combined view `vw_dq_all_orphans` unions all DQ views into a single table for Power BI reporting

```sql
CREATE VIEW "vw_dq_dungeon_game_orphans" AS
SELECT ldg.Dungeon_ID, ldg.Game_ID, 'Dungeon has no valid Game link' AS DQ_Issue
FROM "link_dungeon_games" ldg
LEFT JOIN "silver_games" sg ON sg.Game_ID = ldg.Game_ID
WHERE sg.Game_ID IS NULL;
```

	Dungeon_ID	Game_ID	DQ_Issue
321	5f6d158a3dd55d9ed2d1c1b2	5f6ce9d805615a85623ec2c1	Dungeon has no valid Game link
322	5f6d158a3dd55d9ed2d1c18b	5f6ce9d805615a85623ec2c9	Dungeon has no valid Game link
323	5f6d158a3dd55d9ed2d1c190	5f6ce9d805615a85623ec2ce	Dungeon has no valid Game link
324	5f6d158a3dd55d9ed2d1c1bc	5f6ce9d805615a85623ec2c9	Dungeon has no valid Game link
325	5f6d158a3dd55d9ed2d1c195	5f6ce9d805615a85623ec2c9	Dungeon has no valid Game link
326	5f6d158a3dd55d9ed2d1c19a	5f6ce9d805615a85623ec2c0	Dungeon has no valid Game link
327	5f6d158a3dd55d9ed2d1c1c6	5f6ce9d805615a85623ec2c1	Dungeon has no valid Game link
328	5f6d158a3dd55d9ed2d1c19f	5f6ce9d805615a85623ec2b7	Dungeon has no valid Game link
329	5f6d158a3dd55d9ed2d1c19f	5f6ce9d805615a85623ec2bd	Dungeon has no valid Game link
330	5f6d158a3dd55d9ed2d1c19f	5f6ce9d805615a85623ec2b8	Dungeon has no valid Game link
331	5f6d158a3dd55d9ed2d1c19f	5f6ce9d805615a85623ec2ba	Dungeon has no valid Game link

Power BI

DAX Measures

Person on most games:

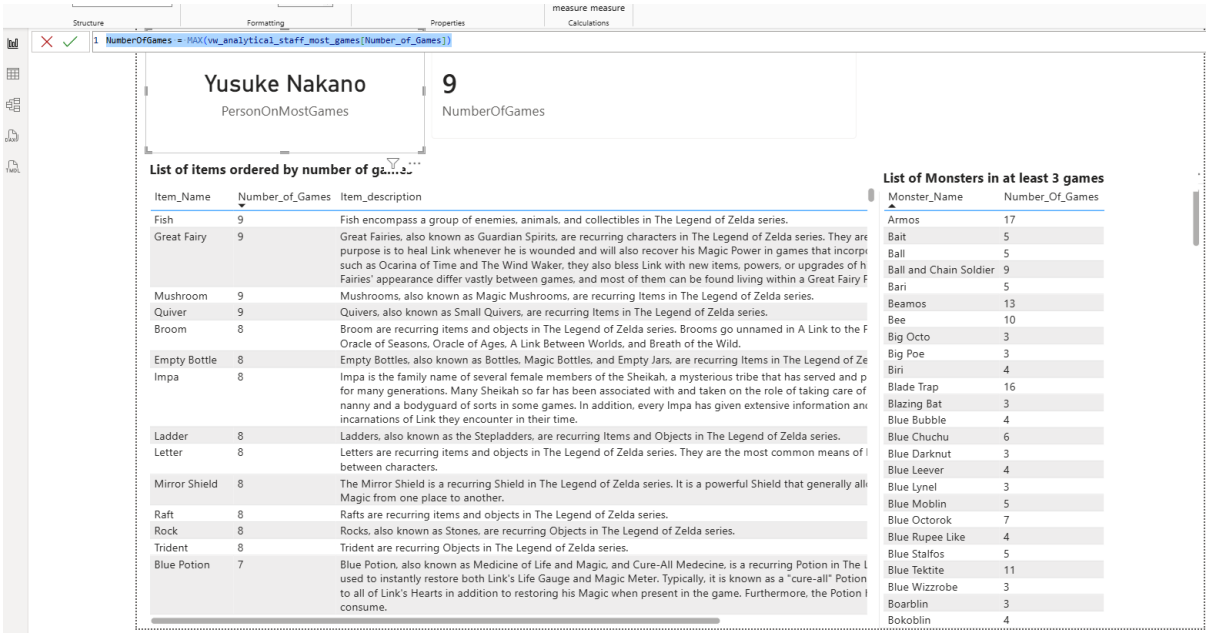
dax

```dax
PersonOnMostGames =
VAR MaxVal = MAX(vw_analytical_staff_most_games[Number_of_Games])
RETURN
CALCULATE(
    FIRSTNONBLANK('vw_analytical_staff_most_games'[Staff_Name], 1),
    'vw_analytical_staff_most_games'[Number_of_Games] = MaxVal
)
```

Number of games:

dax

```dax
NumberOfGames = MAX(vw_analytical_staff_most_games[Number_of_Games])
```



This answers questions:

1. Which staff member worked on the most games?
2. List the monsters that appear in at least 3 games.
3. List items with their descriptions ordered by the number of games in which they appeared, starting with the highest number of games.

Future Considerations

- **Watermarking** — Use an updated/created date from the API to only extract and stage records changed since the last run. Current API version does not support date-based filtering so full extract is performed each run
- **Ghost records** — Add a ghost/unknown record to each silver table so link tables always have a valid record to connect to even where no matching entity was found in the source
- **Staff PII separation** — Create a separate Staff PII table containing personal information (name, gender) so Staff analytics can be performed using only Staff_ID without exposing personal data where not required
- **Extended DQ rules** — Current DQ covers cases where a linked ID exists in a link table but has no matching silver record. Future rules could additionally surface null values in source staging tables e.g. bosses with no game appearances at all
- Dataflows to execute python scripts and refresh PBI in a pipeline
- Recorded 'logs' in pbi for maintenance and performance reporting
- Change File Structure and db location to read 'ZeldaDatabase.DB' from within project to not require full filepath